

Lab preparation:

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/465_HW2$ python3 --version
Python 3.12.3
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/465_HW2$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/465_HW2$ pip show cryptography pytest
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives to Python developers.
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors <cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
---
Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laughner, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/465_HW2$
```

Did not make modifications to the Lab Preparation within the instructions document

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/465_HW2$ openssl genpkey -g
enparam -algorithm DH -pkeyopt group:ffdhe3072 -out ffdhe3072.pem
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/465_HW2$ openssl dhparam -i
n ffdhe3072.pem -text -noout | head -3
    DH Parameters: (3072 bit)
    GROUP: ffdhe3072
```

ffdhe3072.pem generated within 465_HW2 directory

Task 1:

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/465_HW2$ python3 baseline_c
tr.py
Output of original plaintext: {'action': 'READ', 'path': 'notes.txt'}
Output of modified plaintext: {'action': 'LIST', 'path': 'notes.txt'}
XOR between original and modified ciphertext: b'\x00\x00\x00\x00\x00\x00
\x00\x00\x00\x00\x00\x00\x1e\x0c\x12\x10\x00\x00\x00\x00\x00\x00\x00
\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00'
Replay of original plaintext: {'action': 'READ', 'path': 'notes.txt'}
Replay of modified plaintext: {'action': 'LIST', 'path': 'notes.txt'}
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/465_HW2$
```

In my design, I created encryption and decryption functions to easily allow for replay when needed. The plaintext defined in the instruction document

`{"action":"READ","path":"notes.txt"}` was turned to bytes, encrypted using a random key and initialization vector, then encrypted using the AES-CTR function of the cryptography library. After creating the ciphertext, I created a copy with the action modified from READ to LIST to match the number of bytes, then did an XOR operation with the original plaintext and ciphertext in the relay function to simulate an attack intercepting and modifying ciphertext. As shown by printing out the XOR between the original and modified ciphertext, only four bytes have changed due to changing from LIST to READ and the decryption still works with the modified ciphertext using the same key and IV. After decrypting both the original and modified ciphertext again using the same parameters, I simulated a replay attack which would not be caught.

The root cause of it being easy to modify the ciphertext is that the only thing required for generating the ciphertext is the output of the key and IV from the encryption function XOR'd with the plaintext message. As such, we can find the bits to XOR against the ciphertext, which is simulated here by performing an XOR operation between the original and modified plaintext. Taking the output of that operation and performing XOR against the ciphertext will produce a ciphertext that, when decrypted, will output the modified message. As for replay attacks, there is no variable tracking if ciphertext has already been used, making it possible to decrypt the same ciphertext without knowing the key or IV.

Task 2:

After loading the pem file, the values from it are taken as parameters to establish the Diffie-Hellman keys for both parties. These are then exchanged, then fresh nonces are created for each session to add replay resistance. Next, a canonical transcript is formed containing the protocol label, group identifier, party identities, the ephemeral values for both parties, and the nonces for both parties. The ephemeral values are important here as it allows for a new shared secret to be generated each time, ensuring forward secrecy if the keys are stolen. The values of the transcript are checked before being signed to prevent verification of bad data. After passing the check, the transcript is hashed and concatenated with the roles of each party, then given a digital signature which is then verified to perform identity authentication. Afterwards, new keys for encryption and message authentication codes are created by concatenating specific identities with the hashed transcript and calculating hash-based message authentication codes to provide key separation.

By signing the identities, roles, nonces, and Diffie-Hellman values, we can check for man-in-the-middle attacks since any modification will lead to a different signature than expected by the public key. Reflection attacks are also prevented because, if we have sender A and receiver B, sender A is expected to verify with B's public key. If a message sent by A is reflected back, they will know that it has not been sent by B because B's public key will not verify the message. Without signatures, man-in-the-middle and reflection attacks are possible since there is no way to authenticate the entity who sent the message or if it has been tampered with. As such, it is possible to receive messages from attackers who have intercepted the sent message and made some sort of modification.

Task 3:

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/465_HW2$ python3 secure_record.py
b'Messaging gateway'
b'Messaging node'
```

In this task, the keys are received from handshake.py, with each set of encrypting and message authentication code keys given a specific direction (0 for node to gateway, 1 for gateway to node). When communicating, the parameters for sealing are the associated keys, session_id from handshake.py, tracker for number of nodes sent, the message type, and plaintext. The direction is then checked against the keys and an IV is created using a concatenation of the session_id and current sequence. These are encrypted using AES-CTR to create ciphertext, then creating a header that concatenates the version number, direction, sequence, message type, and length of the ciphertext to prevent modification to ciphertext of a different length. The header concatenated with the IV and ciphertext is turned into an HMAC tag, with the record formed by the header concatenated with the ciphertext and tag.

When opening the record from the sender, the parameters are the record, associated keys, session_id, expected direction, and expected sequence number. These are checked against the parameters in the record and verified against expected values and the tag. If there are no errors, the ciphertext within the record is decrypted and turned into plaintext for the receiver to access. Afterwards, the expected sequence from the sender is incremented to prevent replay attacks.

Task 4:

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/465_HW2$ python3 test.py
Handshake and message send successful
Detected ciphertext modification
Detected header modification
Detected replay attempt
Attempt to modify direction: Unexpected direction
Signature error
Incorrect signature
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/465_HW2$
```

Tested `secure_record.py` for correct function (handshake and message), modifying ciphertext in `open_record`, modifying header in `open_record`, replaying a record in `open_record`, reflecting back a record and using `open_record`, and creating a modification of `perform_handshake` with user substituted signatures.

Security Note:

In `baseline_ctr.py`, I assume that the attacker knows both the plaintext and the ciphertext. It is possible that, if knowing only the pattern of the plaintext, that they could bruteforce and achieve the same result but this worked for the simulation. For the tests, which involved `handshake.py` and `secure_record.py`, I assume that the hacker is able to view and modify the ciphertext and knows the general structure of the code, attacking between two endpoints to attempt to modify messages or reflect back messages, along with the general structure of what the messages in attempting to modify the record. Non-goals for this model are disrupting the communication between the node and gateway. For the attack to work, the messages must be intercepted and modified or handled in another malicious way. Breaking the link or causing unreasonable delay would cause this to fail or be easily detected, respectively.

Counter needs unique IVs because otherwise a given plaintext will always result in the same ciphertext, which gives the attacker more hints on what the potential message is. For counter mode, the IV along with the key is XORed with the ciphertext to obtain the plaintext. If ciphertexts from two encryptions with the same key (which usually does not change) and IV are XORd with each other, the output is the XOR of the two corresponding plaintexts. From there, the attacker can analyze the XOR and figure out what each message is directly.

Verifying before decrypting matters because it is unknown what could happen if a modified message is decrypted. In this case, sending a short message is unlikely to cause damage to a system, but it is possible that error states give the attacker more information on what the plaintext is.

Having different keys for encryption and MAC is to ensure the key separation protocol is being followed. Encryption and MAC perform different functions and although there is a bit more work in keeping track of several keys, it is worth it so that attackers cannot undo encryption if the MAC keys are leaked or vice-versa.

In AEAD, the encryption of AES is combined with tagging that allows for authentication. In the seal function, the plaintext is encrypted using AES-CTR, then added to a record along with a header and a tag. In the open_record function, both the record and the tag are verified against the data sent in as parameters. If this process is successful, the ciphertext is decrypted and the plaintext is returned. This is the general structure of AES-GCM, which uses AES-CTR encryption and adds a tag that is verified to ensure the ciphertext and associated data have not been changed.

It is possible that a record be valid but still contain elements that could cause harm. For example, maybe the data itself is valid but there is extra information that an attacker has added that leads to a tool causing damage to the system or leaking information about the message to the attacker. It is also possible that the tool was intended to be in the record, but only intended to be sent and not to receive and will do harm if reflected. In this case, it is important that authentication of the sender occurs before parsing the record, if possible.